Universität Bremen

Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen Welten-Generation

Bachelorarbeit zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc.) in Informatik

Eingereicht von

Maarten Behn

maarten.behn@uni-bremen.de

Prüfer

Prof. Dr. Gabriel Zachmann
Prof. Dr. Nico Hochgeschwender

FB3

Uni Bremen

SoSe 2026

16. March 2026

1 Einleitung

Prozedurale Welten werden mit Algorithmen generiert. Dabei werden werden erst grobe Eigenschaften generiert und diese dann mit immer feiner werdenden Details ausgeschmückt. Wenn der Generations Algorithmus in irgendeiner form angepasst wird kann es sein das eine vorher generierte Welt nicht mehr dem aktuellen Algorithmus entspricht. Die Rechenzeit eines prozeduralen Algorithmus steigt mit der Menge an Details die generiert wird. Bei komplexen Algorithmen die viele Details generieren, kann die lange Neugenerationszeit einer Welt beim iterativen entwickeln von generations Algorithmen stören.

1.1 Ziel der Arbeit

Ziel dieser Arbeit ist zu erforschen in wie weit die Neugenerationszeit einer prozeduralen Welt verschnellert werden kann, wenn nur die Aspekte der Welt neugeneriert werden die nicht mehr den neuen Algorithmus entspricht. Dazu habe ich ein System entwickelt in welchem der generations Algorithmus als Abhängigkeits-Graph dargestellt wird. Bei einer Neugeneration können zwischen Ergebnisse für Abschnitte des Abhängigkeits-Graph, welche sich nicht veränder haben wiederverwendet werden.

1.2 Aufbau der Arbeit

2 Stand der Technik

2.1 Noise & Zufälligkeit

Ein sehr verbreiteter Ansatz zur prozeduralen Generierung von Welten basiert auf der Verwendung von Noise-Algorithmen. Typischerweise werden hierfür kontinuierliche Noise-Algorithmen wie Perlin-Noise oder Simplex-Noise verwendet. Daher wird die Struktur der generierten Welt nicht durch explizite Regeln beschrieben, sondern ergibt sich aus einer überlagerung meherer Noise Funktionen mit unterschiedlichen Frequenzen und Amplituden.

Noise-basierte Verfahren sind relativ leicht zu implementieren und skalieren gut auf große Welten. Aus diesem Grund werden sie häufig in Spielen und Simulationssystemen eingesetzt, insbesondere zur Generierung von Gelände, Vegetation oder anderen natürlichen Strukturen.

Ein wesentlicher Nachteil von rein zufallsbasierten Generationsverfahren besteht darin, dass komplexe strukturelle Regeln nur schwer garantiert werden können. Da die schwer vorher zu sagen ist wie die Noise Funktionen in sonder Fällen zusammen spielen, ist es schwierig sicherzustellen, dass bestimmte globale oder semantische Anforderungen immer erfüllt werden.

Dies kann dazu führen, dass in seltenen Fällen unerwünschte oder fehlerhafte Strukturen entstehen, beispielsweise unzugängliche Bereiche, unnatürliche Geländeformationen oder inkonsistente Weltstrukturen. Solche Probleme lassen sich häufig nur durch zusätzliche heuristische Korrekturen oder nachträgliche Validierungsschritte beheben.

Ein weiterer Nachteil im Kontext dieser Arbeit ist, dass die Regeln der Generierung implizit in der komposition der Noise Funktionen enthalten ist. Wenn der Algorithmus verändert wird, ist es daher schwierig zu bestimmen, welche Teile einer bereits generierten Welt noch gültig sind und welche neu berechnet werden müssen.

2.2 Example based Model Synthesis

Eine spannende Arbeit im Bereich „constraint based generation“ ist Paul Merrels „Example-based Model Synthesis“. [1]

Die Idee des Algorithmus ist aus einem kleinen Input-Datensatz eine größere Struktur zu erzeugen die lokal den gleichen Regeln folgt.

Dabei besteht der Input-Datensatz aus einer Gitter Struktur, wo jede Celle im Gitter ein Wert zu geordnet wird. Nun wird eine Liste aller Nachbar Kombinationen erstellt die im Input Input-Datensatz vorkommen. Diese Liste beschreibt die Regeln nachdem ein neues Modell erzeugt wird.

Generations Algorithmus:

1. Dabei werden dem Gitter des neuen Modells in jeder Zelle alle Werte zugeordnet.
2. Wähle die Zelle mit den wenigsten Werten aus und entferne alle bis auf einen.
3. Entferne alle Werte aus den Nachbar Zellen für die eine Regel nicht mehr erfüllt ist.
4. Wiederhole 3. für jede Nachbar Zelle bei der Werte entfernt wurden.

5. Wiederhole 2. bis alle Zellen nur noch einen Wert enthalten.

2.2.1 Minimale Neuberechnung mit Model Systhesis

Ein möglicher Ansatz zur partiellen Neuberechnung prozedural generierter Welten könnte auf Verfahren der Example-based Model Synthesis aufbauen. Die grundlegende Idee wäre zunächst eine vollständige Welt mit dem Model-Synthesis-Algorithmus zu generieren. Wenn sich anschließend die zugrunde liegenden Regeln ändern, könnte versucht werden, nur die Teile der Welt zu verändern, die den neuen Regeln nicht mehr entsprechen.

Ein naheliegender Ansatz wäre, alle Felder zu identifizieren, deren aktuelle Werte gegen die neuen Regeln verstoßen. Für diese Felder müssten anschließend wieder mehrere mögliche Werte zugelassen werden, sodass der Model-Synthesis-Algorithmus erneut eine konsistente Lösung finden kann.

Dabei stellt sich jedoch ein grundlegendes Problem: Wenn einem Feld neue mögliche Werte hinzugefügt werden, sind diese zunächst nicht notwendigerweise mit den aktuellen Werten der Nachbarfelder kompatibel. Damit die lokalen Regeln wieder erfüllt sind, müssten auch den Nachbarfeldern zusätzliche mögliche Werte hinzugefügt werden. Dieser Prozess kann sich wiederum auf deren Nachbarn ausbreiten und so weiter.

Würde man dieses Verfahren naiv implementieren, indem für alle betroffenen Nachbarn wieder sämtliche möglichen Werte zugelassen werden, entstünde im Extremfall erneut ein vollständig unentschiedenes Gitter. In diesem Fall würde der Algorithmus effektiv wieder bei einem normalen Model-Synthesis-Prozess beginnen, wodurch kein Vorteil gegenüber einer vollständigen Neugenerierung entsteht.

Um dennoch einen Nutzen aus diesem Ansatz zu ziehen, müsste eine möglichst kleine Menge an Feldern gefunden werden, deren Wertebereiche erweitert werden, sodass anschließend wieder eine konsistente Lösung existiert. Diese Menge sollte idealerweise minimal sein, damit möglichst große Teile der bestehenden Welt unverändert bleiben können. Gleichzeitig müsste der Aufwand zur Bestimmung dieser Menge deutlich geringer sein als eine komplette Neugenerierung der Welt.

Eine theoretische Möglichkeit bestünde darin, den Raum der möglichen Werteerweiterungen systematisch zu durchsuchen. Beispielsweise könnte eine Breitensuche über den Graphen der möglichen Wertkombinationen durchgeführt werden, um eine minimale Menge an Änderungen zu finden, die wieder zu einer gültigen Konfiguration führt. Allerdings wächst dieser Suchraum sehr schnell und führt sowohl in Bezug auf Laufzeit als auch Speicherverbrauch zu erheblichen Komplexitätsproblemen.

Der Vorteil des ursprünglichen Model-Synthesis-Algorithmus liegt darin, dass zu jedem Zeitpunkt alle noch möglichen Kombinationen eine valide Lösung darstellen. Das Finden einer minimalen Erweiterung dieser Mengen, die nach einer Regeländerung wieder eine gültige Lösung ermöglicht, ist jedoch deutlich schwieriger als die ursprüngliche Generierung selbst.

2.3 L-Systems

L-Systems (Lindenmayer-Systeme) sind ein regelbasiertes Verfahren zur Beschreibung und Generierung komplexer Strukturen. Sie wurden ursprünglich zur Modellierung biologischer Wachstumsprozesse entwickelt und werden heute häufig in der prozeduralen Generierung natürlicher Strukturen eingesetzt, beispielsweise für Pflanzen, Bäume oder Korallen.

Ein L-System besteht im Kern aus einem Alphabet von Symbolen, einem Startsymbol, und einer Menge an Produktionsregeln.

Die Produktionsregeln definieren, wie jedes Symbol durch eine Folge anderer Symbole ersetzt wird. Ausgehend vom Startsymbol wird in jedem Iterationsschritt jedes Symbol gemäß den definierten Regeln ersetzt. Dadurch entsteht schrittweise eine immer längere Symbolsequenz. Nach mehreren Iterationen entsteht eine lange Symbolfolge, die anschließend interpretiert werden kann.

Ein wesentlicher Vorteil von L-Systems ist ihre Fähigkeit, mit wenigen Regeln sehr komplexe und selbstähnliche Strukturen zu erzeugen. Besonders fraktalartige Formen und verzweigte Strukturen wie Pflanzen lassen sich damit sehr kompakt beschreiben.

Ein Nachteil von L-Systems besteht darin, dass sie primär auf rekursiven Ersetzungsregeln basieren und daher weniger flexibel für die Beschreibung allgemeiner struktureller Abhängigkeiten sind. Die erzeugten Strukturen folgen strikt den definierten Produktionsregeln, wodurch es schwierig sein kann, globale Einschränkungen oder komplexe Interaktionen zwischen entfernten Teilen der Struktur zu modellieren.

Ein weiterer Aspekt im Kontext dieser Arbeit ist, dass Änderungen an den Produktionsregeln typischerweise eine vollständige Neuberechnung aller Iterationen erfordern. Da jede Iteration auf der vorherigen basiert, können Änderungen an einer Regel potenziell die gesamte erzeugte Struktur beeinflussen. Dadurch eignet sich dieser Ansatz nur eingeschränkt für Szenarien, in denen eine minimale Neuberechnung nach Änderungen des Generationsalgorithmus angestrebt wird.

2.4 Graph based Model Synthesis

Die Arbeit Graph based Model Synthesis verbindet die Idee von Graph Grammatiken und Model-Synthesis.

Die Grundidee besteht darin, dass Generationsregeln nicht direkt auf einer expliziten Geometrie arbeiten, sondern auf dem zugrunde liegenden Graphen angewendet werden. Eine Regel beschreibt dabei eine Transformation eines Teilgraphen in einen anderen Teilgraphen. Durch wiederholte Anwendung solcher Regeln kann ein initialer Graph schrittweise erweitert oder verändert werden.

Ein Beispiel für diesen Ansatz ist die Verwendung von Graph-Grammatiken zur prozeduralen Modellierung von Polygonmodellen. Dabei wird ein bestehendes Modell zunächst in eine Menge kleiner Strukturelemente zerlegt. Aus diesen Elementen wird anschließend eine Menge von Regeln abgeleitet, die beschreiben, wie lokale Graphstrukturen miteinander kombiniert werden dürfen. Durch wiederholte Anwendung dieser

Regeln können neue Modelle erzeugt werden, die lokal ähnliche Strukturen wie das ursprüngliche Beispiel aufweisen.

Ein Vorteil dieses Ansatzes ist, dass komplexe strukturelle Zusammenhänge explizit im Graphen dargestellt werden. Dadurch können Generationsregeln sehr präzise definieren, unter welchen Bedingungen eine Transformation erlaubt ist. Gleichzeitig lassen sich globale Strukturen einfacher kontrollieren als bei rein stochastischen Verfahren wie Noise-basierten Methoden.

Ein Nachteil besteht jedoch darin, dass die Definition geeigneter Graphregeln häufig aufwendig ist und ein gutes Verständnis der zugrunde liegenden Datenstruktur erfordert. Außerdem kann die Anwendung vieler Transformationsregeln auf großen Graphen zu einem hohen Rechenaufwand führen.

2.5 KI basierte prozedurale Generation

Neuronale Netzwerke können genutzt werden um ähnliche Inhalte zu einem Trainings Datensatz zu erzeugen. In Arbeiten wie [2], [3] und [4] werden Neuronale Modelle genutzt um Levels aus bekannten 2D Spielen zu reproduzieren. Dabei wird in [2] wird ein Level generations Model gegen ein Spieler Argent Modell trainiert, welches bewertet ob ein Level spielbar ist. Der Ansatz ist meistens fähig spielbare Levels zu generieren, wobei es vereinzelt zu generation Artefakten kommen kann. Jedoch behandelt die Ansätze simple 2D Spiele, für die es viel vorhandene Level zu trainieren gibt.

2.5.1 Terrain Diffusion

Im Ansatz „Terrain Diffusion“ werden Neuronale-Diffusion-Modelle verwendet. Diffusion-Modelle erzeugen Daten, indem sie iterativ aus zufälligem Rauschen eine strukturierte Lösung rekonstruieren. Dieses Verfahren wird ursprünglich für Bildgenerierung eingesetzt, kann jedoch auch zur Erzeugung von Höhenkarten verwendet werden.

Hier wird ein neuronales Modell auf reale Geländedaten trainiert, sodass es realistische Terrainstrukturen erzeugen kann. Um weiterhin unendliche Welten generieren zu können, wird das Gelände in Kacheln erzeugt, die deterministisch aus einem Seed generiert werden. Dadurch können neue Bereiche der Welt bei Bedarf berechnet werden, ähnlich wie bei klassischen Noise-basierten Verfahren.

Der Vorteil dieses Ansatzes liegt darin, dass größere geographische Strukturen realistischer modelliert werden können als mit klassischen Noise-Funktionen.

Ein Nachteil ist jedoch, dass das Verhalten des Systems stark vom Trainingsdatensatz abhängt und Generationsregeln nicht explizit definiert sind. Dadurch ist es schwierig sicherzustellen, dass bestimmte strukturelle Anforderungen immer erfüllt werden. [5]

2.5.2 Minimale Neuberechnung

Bei KI-basierten Ansätzen zur prozeduralen Generierung ist nicht explizit nachvollziehbar, welche Teile der Eingabe und des Modells zu welchen Teilen des erzeugten Ergebnisses führen. Wenn sich die Generationsregeln oder Eingaben ändern, kann daher

nicht einfach bestimmt werden, welche Teile der bereits generierten Welt weiterhin gültig sind. Daher eignen sich KI-Ansätzen für meine Arbeit nicht.

3 Theoretische Grundlagen

3.1 Minimales errechnen (Lazy computation)

Minimales errechnen auf englisch Lazy computation beschreibt die Idee in Computerprogrammen die Daten erst dann zu errechnen wenn sie benötigt werden.

Es reduziert Lag-spikes gerade beim starten eines neuen Prozesses, da Anstatt alle nutzbaren Daten nur die, die gerade angefragt benötigt werden errechnet werden.

In Funktionale Programmiersprachen wie Haskell findet dieses Konzept vie Anwendung und erlaubt für unendliche Datenstrukturen.

3.2 Graphen

Die Datenstrukturen in dieser Arbeit basieren auf mathematische Graphen.

Daher erklärt dieser Abschnitt die in der weiteren Arbeit genutzten Notationen.

Ein Graph $G := (V, E)$ ist besteht aus einer Menge an Knoten V und einer Menge an Kanten E. Die Funktionen $V(G) = V$ und $E(G) = E$ werden als verkürzte Notationen genutzt.

Ein einzelner Knoten wird als $v \in V$ und eine einzelne Kante als $e \in E$ geschrieben.

Eine Kante ist ein Tuple zweier Knoten $e := (v_a, v_b)$. In dieser Arbeit sind Kanten grundsätzlich gerichtet und werden dann auch als $\overrightarrow{v_a v_b}$ geschrieben.

Die Menge aller eingehenden bzw. ausgehender Kanten eines Knoten v wird als $E_G^-(v)$ und $E_G^+(v)$ geschrieben.

Die Menge aller Knoten die eine eingehende bzw. ausgehende Kante zu einem Knoten haben werden als $N_G^-(v)$ und $N_G^+(v)$ geschrieben, da sie als die Nachbarn von diesem Knoten verstanden werden.

Diese Notationen basieren auf den Notationen in „Modern Graph Theory“ von Béla Bollobás [6].

3.3 Constructive Solid Geometry (CSG)

Constructive Solid Geometry (CSG) ist eine Methode zur Darstellung von Volumen, bei der komplexe Geometrien durch die Kombination primitiver Geometry (Box, Kugel, etc) dargestellt wird.

Diese Kombination sind standartmäßig Vereinigung, Schnittmenge und Differenz, wurden jedoch in späteren Arbeiten mit komplexeren Operationen, wie Verformung, Duplikation, etc erweitert.

CSGs werden als directed acyclic graph (DAG) gespeichert. Jede Knoten is eine implizierte Operation auf den Volumen der Kinder. Die Blätter sind dahingegen durch Parameter definierte primitive Geometrien.

Der Vorteil dieser Darstellung ist, dass Geometrie über Parameter und Operationen beschrieben wird. Änderungen an Position, Größe oder anderen Parametern wirken sich

direkt auf das resultierende Volumen aus, ohne dass die gesamte Geometrie neu definiert werden muss.

Jedoch eignen sich CSGs selten zum direkten rendering mit raytracing, da die Operationen zu leistungsaufwendig sind um mit jeden Ray den DAG zu rekursiv zu iterieren. Dazu steigt die Größe des DAGs schnell mit der Komplexität des CSGs. [7], [8]

3.4 Grafische-Programmierung

Grafische-Programmierung ist eine From der Programmierung bei der Logik nicht als Code definiert ist sonder als Diagram in einem grafischen Editor. Funktionen werden als Knoten mit Input und Output dargestellt. Dies können mit Linien verbunden werden um logische Abfolgen zu definieren.

Grafische-Programmierung erlaubt ein besseren überblick über Programmabschnitte und ist durch seinen invitieren Syntax Einsteigerfreundlicher.

Grafische-Programme ausgeführt werden indem die Operationen als Graph dargestellt werden. Dieser kann vergleichbar zu Funktionenprogrammiersparchen rekursiv gelöst werden.

Jedoch ist es auch möglich einen Grafische-Programme in Code zu übersetzen und Maschinen-Code zu kompilieren. Dies kann massive Leistungsvorteile haben.

Populäre Programme in den Grafische-Programmierung verwendet wird, sind Unity Shader Graphs, Unreal Engine Templates und Blender Geometry Nodes.

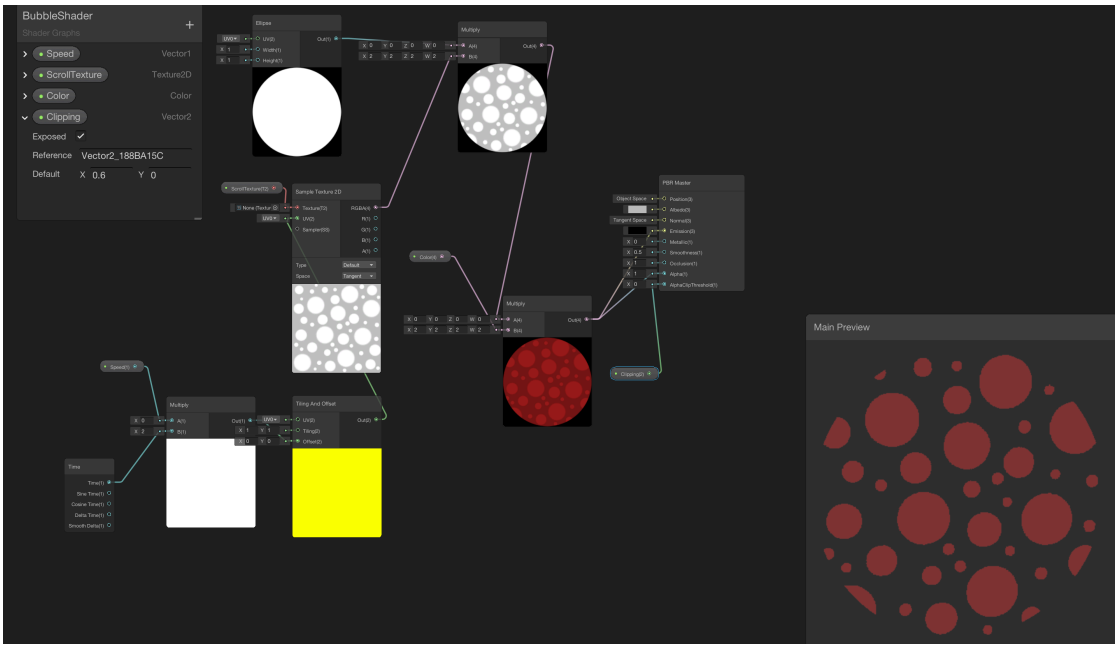

Abbildung 1: Der grafische Editor von Unity Shader Graph

4 Meine Arbeit

4.1 Kern Idee

Die Idee ist das die gesamte zu generierende Welt als Graph an Operationen definiert. Operationen errechnen aus eine Menge an Input Werten in eine Menge an Output Werten.

Beispiel für Operationen sind:

- Addition
- Erzeugung einer Kugel aus Position und Größe
- Union zweier Volumen
- etc

Operationen nutzen die Ergebnisse von anderen Operationen und bilden so ein Graph an Abhängigkeiten.

Bei der Errechnung werden die Ergebnisse der Operations Knoten zwischen gespeichert.

Wenn sich nun der Abhängigkeits-Graph minimal ändert entspricht das vorherige Ergebnis potenziell nicht mehr dem Abhängigkeits-Graph. Dies verstehe ich in dem Rest der Arbeit als eine invalide Lösung/Ergebnis.

Jedoch muss durch die zwischen Speicherung nicht den ganze Graph neu errechnet werden, sondern nur den Teil der nicht mehr valide ist.

Die Kern Leistung meiner Arbeit ist die Entwicklung einer effizienten Implementierung für dieses Modell.

4.2 Abhängigkeits-Graph

Jede mathematische Formel oder Algorithmus kann als Graph an Abhängigkeiten dargestellt werden. Dabei beschreibt jeder Knoten eine mathematische Operation die aus ein oder mehren Eingangswerten einen Ausgangswert errechnet. Die Eingangswerte eines Knoten sind die Ergebnisse andere Knoten im Graph. Dies bezeichne ich als Abhängigkeit, da die Ergebnisse der Eingangsknoten für die Berechnung benötigt werden. Das Ergebnis eines Knoten hängt nur von den Eingangswerten ab und hat damit keine Nebeneffekte. (siehe Funktionale Programmierung) Knoten können keine Eingangswerte haben dann erzeugen diese immer die gleichen Werte und werden als konstant verstanden.

4.3 Framework

ToDo:

Overview diagramm

Mein generations System besteht aus drei Bestandteilen.

1. Ein Graphischer Editor indem ein Nutzer einen Abhängigkeits-Graph erstellen und bearbeiten kann.

2. Das Template ist eine Datenstruktur die vom Editor erstellt wird und den Abhängigkeits-Graph sowie Informationen wie dieser generiert und zwischen gespeichert werden soll enthält.
3. Der Generator vergleicht das aktuelle Template mit dem neuen Template und generiert die Bestandteile der Welt neu die nicht dem neuen Template entsprechen.

4.4 Composer

Zur interaktiven definition des Abhängigkeits-Graph wird ein Grafische-Programmierungs-Editor vergleichbar zu Unreal Templates, Blender Geometry Nodes oder Unity Shader Graph genutzt.

Der Nutzer kann Knoten erstellen die einer Operation entsprechen und frei auf einer unendlichen Fläche angeordnet werden können. Diese Operationen haben auf der linken Seite eine Liste an Input Variabeln und an der Rechten Seite eine Liste an Ergebnissen.

ToDo:

Bild von Node

Die Variabeln und Ergebnisse sind farbig nach ihrem Datentyp kodiert und können mit Linien verbunden werden. Die Symbolsiert, das ein Ergebnis als Variabele für eine andere Operation verwendet werden soll.

So können komplexe Algorithm über Knoten Verbindungen von links nach rechts dargestellt werden, parallele Stränge werden übereinander angeordnet. Die fördert eine Darstellung die komplexe Abhängigkeiten übersichtlich darstellen kann.

ToDo:

Bild von meheren Knoten

4.5 Template

Das Template $:= (G_{\text{ab}}, G_{\text{ch}})$ besteht aus dem Abhängigkeits-Graph G_{ab} und einem „cache“ Graphen G_{ch}.

G_{ab} ist ein Graph der die zu generierende Welt als rekursive Formel an Operationen beschreibt.

Die ein gehenden Nachbarn $N^{-}_{G_{\text{ab}}}$ errechnen die Eingangswerte für eine Operation und die ausgehenden Nachbarn $N^{+}_{G_{\text{ab}}}$ sind alle Operationen die das Ergebnis benötigen.

ToDo:

Beispiel?

G_{ch} enthält einen Knoten für jeden Knoten in G_{ab} der Zwischengespeichert werden soll. Dies ist eine Untermenge aller Knoten in G_{ab}.

$V(G_{\text{ch}}) \subset V(G_{\text{ab}})$

Bei der Entscheidung wie groß diese Untermenge sein soll muss zwischen dem „Overhead“ durch zwischengespeicherung und der Zeitersparniss durch wiederverwendung der Ergebnisse abgewägt werden.

Die ein gehenden Nachbarn $N^-_{G_{\text{ch}}}$ sind alle caches von dem der Knoten abhängt. Man findet diese indem man den Baum der Abhängigkeiten in G_{ab} über alle Verzweigungen rekursiv durchsucht bis man auf jeden Weg einen auf einen Knoten in G_{ch} stößt.

ToDo:

Algorithmus angeben? eigentlich ist das nicht wichtig.

ToDo:

Grafik von Datentypen und Abhängigkeits-Graph

4.5.1 Level

Wenn der G_{ab} ein directed acyclic Graph (DAG) ist dann kann jedem Knoten $v \in G_{\text{ab}}$ ein Level $l(v)$ zugeordnet werden. Dies ist definiert als:

$$l(v) > l(v_i) \quad \forall v_i \in N^-_{G_{\text{ab}}}(v)$$

Also das Level eines Knoten ist immer größer als das Level aller Knoten von den er abhängt.

Um den G_{ab} zu errechnen werden die Knoten der Level aufsteigend errechnet. Die Rheinfolge innerhalb eines Levels ist nicht relevant.

4.5.2 Einen Prozeduralen Algorithmus als Template darstellen

Bis zu diesem Abschnitt habe ich das Template sehr allgemein beschrieben.

Um mit dem Template eine Prozedurale Welt zu generieren haben ich mich entschieden, dass das Template final eine Volumen als „constructive solid geometry“ (CSG) errechnet.

Diese CSG setzt sich durch union und remove Operationen auf primitiven Geometrien wie Kugeln und Boxen zusammen.

Daher sind die Operationen die ich implementiert habe:

- CSG Union und Remove
- Kugel aus Position und Durchmesser
- Box aus Position und Seitenlänge
- Alle Positionen auf einem Gitter die innerhalb eines Volumen sind.
- Eine Menge an zufälligen Positionen innerhalb eines Volumen. siehe: genauer dazu
- Addition, Subtraktion, Multiplikation und Division von Positionen und Zahlen
- etc.

4.5.3 Generation eines Templates

Operationen im Abhängigkeits-Graph G_{ab} können auch Mengen an Werten erzeugen. In meiner implementation ist das die beiden Operationen die ein Gitter und zufällige Positionen in einem Volumen errechnen.

Nun kann es sein, dass die weiteren Operationen pro Element in dieser Menge ausgeführt werden sollen. Dies ist der Bestandteil von prozeduraler Generation der es möglich immer feinere Details zu generieren. Zu Beispiel errechnet man eine Menge an Positionen an den Apfelbäume stehen sollen und dann generiert man pro Baum die Positionen der Äpfel.

Dies ergibt einen klaren Unterschied in der Laufzeit von Algorithmen auf dem Template zu Algorithmen auf der generierten Welt. Die Menge an Knoten im Abhängigkeits-Graph und so auch den cache Graph skaliert mit der Menge an Operationen des Generationsalgorithums. Wohin die Menge der Elemente in der Welt mit dem Größen der Mengen an rechneten Werten skaliert. In anderen Worten alle Knoten im Template zu iterieren ist relativ schnell möglich wohin gegen die Laufzeit alle Elemente in der Welt zu iterieren exponentiell steigen kann.

Daher arbeiten alle Algorithmen im meinen System immer nur auf den Knoten des Templates. Sie nutzen die Abhängigkeiten im Template um heraus zu finden wie die Welt neu generiert werden muss.

4.6 Generator

Der Generator enthält einen Graphen G_{gen} der den cache Graphen G_{ch} im Template entspricht. Jeder Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ speichert welchem Knoten $v_{\text{ch}} \in V(G_{\text{ch}})$ er entspricht $v_{\text{ch}} = \textbf{cache}(v_{\text{gen}})$. Dazu hat ein Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ das Level des entschieden Template Knoten $l(v_{\text{gen}}) = l(\textbf{cache}(v_{\text{gen}}))$.

Jedoch wo G_{ch} nur einen Knoten pro Operation enthält, enthält G_{gen} einen Knoten pro Ergebnis welches errechnet werden muss.

ToDo:

Beispiel

Dazu hält der Generator $:= (G_{\text{gen}}, Q_{\text{tasks}})$ eine Queue Q_{tasks} die zwei Arten Aufträge auf G_{gen} nach ihren Level sortiert.

$$\text{pop}(Q_{\text{tasks}}) := \min_{q \in Q_{\text{tasks}}} (l(q))$$

Errechnungs-Aufträge errechnen den das Ergebnis eines Knoten in G_{gen} und Kind-Update-Aufträge erzeugen oder löschen Kinder sodass sie dem Template entsprechen.

4.6.1 Abhängigkeite Werte im Generator Graph finden

Um einen Knoten in G_{gen} zu errechnen benötigt man die Ergebnisse der Knoten in G_{gen} von den dieser Knoten abhängt. Wie ich in Abschnitt 4.5.3 erklärt habe ist es Laufzeittechnisch nicht möglich diese mit z.B. einer Tiefensuche zu suchen.

Stattdessen wird für jeden Template Knoten $v \in V(G_{\text{ch}})$ einer der Knoten von den er abhängt $N^-_{G_{\text{ch}}}(v)$ als Erstellungs-Knoten $v_c \in V(G_{\text{ch}})$ im Template markiert $v_c = \textbf{create}(v)$.

Um nun für einen Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ alle anderen Knoten zu finden von er abhängt $N^{-}_{G_{\text{gen}}}(v_{\text{gen}})$, werden die relative Schritte in G_{ch} vom Erstellungs-Knoten v_c zu den anderen Abhängigen Knoten als Baum gespeichert $T_{\text{rel}}(v_{\text{gen}})$.

Ein relativer Schritt v_{step} gibt entweder an das man von einem Knoten $v \in V(G_{\text{ch}})$ hoch ($\mathbf{up}(v_{\text{step}})$ = True) in einen Knoten $v_{\text{up}} \in V(G_{\text{ch}})$ von dem v abhängt ($v_{\text{up}} \in N^{-}_{G_{\text{ch}}}(v)$) oder runter ($\mathbf{up}(v_{\text{step}})$ = False) in einen Knoten $v_{\text{down}} \in V(G_{\text{ch}})$ der von v abhängt ($v_{\text{down}} \in N^{+}_{G_{\text{ch}}}(v)$) gehen soll.

Da ein Knoten $v \in V(g_{\text{ch}})$ mehr als einen eingehenden oder ausgehenden Nachbarn haben kann speichert ein relativer Schritt auch in welchen Nachbarn gegangen werden soll ($\mathbf{cache}(v_{\text{step}})$) und dazu ob dieser Nachbar eine Abhängigkeit für v_{gen} ist ($\mathbf{deps}(v_{\text{step}})$ = True).

Jeder relative Schritt v_{step} gibt speichert auch in welche der Nachbar Knoten gegangen werden soll.

Diese relativen Schritte nutzen nur Knoten die ein kleineres Level haben als v_{gen}. Da im Generator Knoten im Level aufsteigend erstellt werden, ist so sichergestellt das alle relativen Wegen existieren.

Für einen Knoten im Template kann es mehrere Knoten im Generator geben, daher können pro Abhängigkeit eines cache Knoten auch mehrere Knoten im Generator gefunden werden.

Algorithm 1: Finde abhänige Knoten in G_{gen}

```
procedure FindDeps(v_gen, v_gen creates)
    T ← T_rel(v_gen)
    D ← ∅
    Q ← ∅
    push(Q , (v_root ∈ T, v_gen creates))

    while Q ≠ ∅ do
        (v_step, v_gen) ← pop(Q)

        if deps(v_step) then
            push(D , v_gen)
        end

        for v_child step ∈ N⁺_T(v_step) do
            N ← N⁻_G_gen(v_gen) if up(v_child step) else N⁺_G_gen(v_gen)

            for v ∈ N do
                if cache(v_child step) = cache(v) then
                    push(Q, (v_child step, v))
                end
            end
        end
    end
    return D
end
```

ToDo:

Beispiel Zeichung

4.6.2 Kind-Update-Aufträge

Kind-Update-Aufträge enthalten den Index des Erstellungs-Knoten und den Index eines Erstellungs-Eintrag $E_{\text{create}}(v_{\text{ch}})$ in dessen Template-Knoten. Dieser Erstellungs-Eintrag definiert wie viele Kinder es geben soll $\mathbf{num}\big(v_{\text{ch}}, v_{\text{gen creates}}\big)$. Dies sind entweder es genau n pro Erstellungs-Knoten oder, oder hängen von dem Wert des Erstellungs-Knoten $v_{\text{gen creates}}$ ab wie z.B. einer Positions Menge. Dazu gibt $\mathbf{valid}\big(v_{\text{gen}}, v_{\text{gen creates}}\big)$ an ob ein Kind v_{gen} noch für den Erstellungs-Knoten $v_{\text{gen creates}}$ noch valide ist. z.B. ob eine Position noch in der Menge an Positionen ist.

Darauf wird die vorhandene Menge an Kindern mit der gewünschten Menge verglichen und bei Ungleichheit neue Kinder Knoten erzeugt oder gelöscht.

Wenn eine neuer Knoten erzeugt wird werden mit dem Baum an relativen Schritten die Indexe von allen abhängige Knoten gesucht und im Knoten gespeichert.

Algorithm 2: Kinder updaten

```
procedure UpdateChild(v_gen, v_ch child)

    C ← { v ∈ N⁺_{G_gen}(v_gen) | cache(v) = v_ch child }

    C_to delete ← { v ∈ C | ¬ valid(v, v_gen) }

    for v ∈ C_to delete do

        Delete(v)
    end

    n ← num(v_ch child, v_gen)
    i ← 0
    for i < n − |C \ C_to delete| do

        Create(v_ch child, v_gen)
    end
end

procedure Create(v_ch, v_gen creates)
    push(V(G_gen), v_new)

    cache(v_new) ← v_ch

    D ← FindDeps(v_new, v_gen creates)

    N⁻_{G_gen}(v_new) ← D

    for v ∈ D do
        push(N⁺_{G_gen}(v), v_new)
    end

    pushCalculate(Q_task, v_new)
end

procedure Delete(v_gen)

    for v ∈ N⁻_{G_gen}(v_gen) do

        remove(N⁺_{G_gen}(v), v_gen)
    end

    for v ∈ N⁺_{G_gen}(v_gen) do

        Delete(v)
    end
    remove(V(G_gen), v_gen)
end
```

4.6.3 Errechnungs Aufträge

Errechnungs Aufträge errechnen den Wert von einem Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ neu. Dabei wird der Knoten im Abhängigkeits-Graph rekursiv errechnet.

Wenn ein der Algorithmus auf einen Knoten $v_{\text{ab}} \in V(G_{\text{ab}})$ stößt, welcher ein cache Knoten hat $v_{\text{ab}} \in V(G_{\text{ch}})$ werden die Werte der jeweiligen abhängigen Knoten von v_{gen} verwendet.

ToDo:

Beispiel

4.6.4 Abhängigkeits Kreise

Das Template kann Abhängigkeits Kreise enthalten. Um trotzdem eine valide Lösung errechnen zu können muss es für jeden Knoten v_{val} einen validen Null Wert geben.

So kann der Abhängigkeits Graph iterativ gelöst werden. Dazu werden pro Kreis im Abhängigkeits-Graph eine Kante als durchgeschnitten markiert $N^{+}_{\text{cut}}(v) \subseteq N^{+}_{G_{\text{dep}}}(v) \quad v \in V\big(G_{\text{dep}}\big)$. Der Abhängigkeits-Graph ohne die durch geschnittenen Kanten $N^{+}_{\text{not cut}}(v) := N^{+}_{G_{\text{dep}}}(v) \setminus N^{+}_{\text{cut}}(v)$ ist ein DAG. Also kann jedem Knoten ein Level $l(v)$ zu geordnet werden.

$$l(v) > l(v_i) \quad \forall v_i \in N^{+}_{\text{not cut}}(v)$$

Die Knoten werden Level nach Level erzeugt und so sichergestellt das alle nicht geschnittenen Abhängigkeiten schon errechnet worden sind, wenn der Knoten selbst errechnet wird. Hat ein Knoten geschnittene Abhängigkeiten werden diese zu Errechnung genutzt wenn sie existiert. Andernfalls wird der Null Wert verwendet. Jeder Knoten der Null Werte für seine geschnittenen Abhängigkeiten nutzt wird nochmal errechnet, sobald alle Knoten einmal errechnet wurden. Dies wird so lange wiederholt bis keine Null Werte mehr verwendet worden sind.

4.7 Implementation

Für die Beispiel Implementation habe ich Rust als Sprache gewählt, da sie erlaubt speichersicheren low level code zu schreiben um die Laufzeit von Algorithm effektiv zu verbessern und ein in Gegensatz zu C++ ein umfassenden leicht zu nutzenden Package-Manager hat.

4.7.1 Stabiele Listen

Alle Graphen sind mit stabilen Listen implementiert.

Eine stabile Listen ist eine sich automatisch vergrößerndes Array. Wenn dass aktuelle Array voll ist wird ein größeres Array alloziert und die werte mit einer memcopy Operation rüber kopiert. Dazu änderen sich die Indexe von Elementen in stabilen Listen nicht. Wenn ein Element entfernt wird, werden die weiteren Elemente nicht aufgeschoben um die Lücke zu schließen, stattdessen wird der Index des nächsten freien Element gespeichert. Dazu speichert die List speichert den Index des ersten freien Element, welches genutzt wird wenn ein neues Element eingefügt wird. Der dort gespeicherte nächste freie Index wird dann als erster freier Index gespeichert. Dies erlaubt Einfüge-, Entfernungs- und Zugriffslaufzeit von $O(1)$.

Dazu wird pro Element eine Versionsnummer gespeichert. Bei einem Zugriff wird die Version des Index mit der Version des Elements an der Stelle des Index verglichen. So bleiben Indexe valide solange das Element in der Liste ist und es wird erkannt wenn man mit veralteten Index zugreift.

Die oberen 32Bit des Indexes werden genutzt um Versionsnummer zu speichern. Somit können in einer stabilen List 2^{32} Elemente gespeichert werden.

Stabiele Listen erlauben Graphen effizient als Listen darzustellen, indem in den Knoten die Indexe der anderen Knoten gespeichert werden zu den Kanten existieren. Stabiele listen sind dabei wesentlich schneller als HashMaps.

ToDo:

Quelle

4.7.2 Multi Threading

Der Collapser sowie die Sampeling Operationen werden in asynchronous Workern ausgeführt. Zur Kommunikation werden Channel verwendet.

ToDo:

Quelle

Der Composer läuft im Render Thread und errechnet bei jeder Änderung das aktuelle Template. Dieses Template wird über ein Channel zum Collapser gesendet. Wenn der Channel noch ein altes Template enthält werden die Änderungen Notizen des alten Template zum neuen hinzugefügt und das Template wird ersetzt.

Die errechnete CSG Darstellung der Welt wird mit einem weiteren Channel an die Sampler gesendet, welche dieses in ein Voxel DAG oder Mesh umrechnet und auf die GPU gelanden.

Wenn die neue Welt hochgenanden wurde wird der Render Thread über ein Channel über diese Änderung informiert.

4.7.3 Small Vectors

Für die Zwischenspeicherung der Werte werden Small Vectors verwendet. Diese haben die Eigenschaft, dass die ersten N Elemente direkt auf dem Stack alloziert werden. Erst wenn diese voll sind wird ein Array auf dem Stack alloziert.

Da alle Werte müssen als Liste behandelt werden, da ein Knoten immer von mehreren Knoten für ein Input abhängt. Jedoch enthält diese Listen meist doch nur ein Element. Small Vectoren erlauben für diese Fälle die Stack allozieren zu sparen und bieten bessere Cache Lokalität.

4.7.4 Output Datenstruktur

Das Kernconzept, einen sehr großen Graphen der eine prozeduralen Algorithmus darstellt, zu bearbeiten, indem man die Abhängigkeiten in einem vergleichbar kleinen Template nutzt, enthält keine konkreten Anahmen über die Art der Geometrie. In

meiner Implementation habe ich CSG genutzt, da es einer sehr allgemeine Form ist, Volumen darzustellen und diese leicht zu bearbeiten sind.

Gerade CSGs mit vielen Knoten sind jedoch nicht performant zu rendern. Deshalb können die CSGs in meiner Implementation entweder als Sparse Voxel DAGs oder mit marching cubes gesampled werden.

4.8 Minimales Beispiel

in welchem lazy generation einen klaren Vorteil hat.

ToDo:

Diagram

4.9 Nutzungs Beispiele

ToDo:

Bilder

- Welt editor
- Infitie World Generation
- LOD systems

5 Bewertung

Die Bewertung eines Systems zur prozeduralen Generierung ist nicht trivial, da unterschiedliche Systeme häufig unterschiedliche Ziele verfolgen. Während einige Ansätze primär auf maximale Generationsgeschwindigkeit oder realistische Ergebnisse optimiert sind, liegt der Fokus dieser Arbeit auf der effizienten Neuberechnung nach Änderungen am Generationsalgorithmus. Ziel ist es, bei kleinen Änderungen am Generationsprozess möglichst große Teile der bereits berechneten Welt wiederverwenden zu können.

Um die Reduktion der Neuberechnungszeit zu bewerten wird untersucht, wie stark sich die Laufzeit im Vergleich zu einer vollständigen Neugenerierung reduziert. Dazu werden im mehren Beispiel Welten die Laufzeit bei 20%, 50%, 80% oder 100% Neuberechnung gemessen.

Ein weiteres wichtiges Kriterium ist der Overhead des Systems. Da der Generationsalgorithmus nicht direkt als kompilierten Programmcode ausgeführt wird, sondern als Abhängigkeitsgraph interpretiert wird, entstehen zusätzliche Kosten. Diese entstehen insbesondere durch:

- die Verwaltung und Modifikation der Graphstruktur,
- das rekursive Auflösen der Abhängigkeiten während der Berechnung.

Ein klassischer, direkt implementierter Generationsalgorithmus könnte stärker durch den Compiler optimiert werden und benötigt keine zusätzliche Graphstruktur. Daher ist es wichtig zu testen, welchen Anteil der Laufzeit durch zusätzlichen Overhead verursacht wird. Den Laufzeit Anteil der Gartenstruktur lässt sich grob mit einem profiler messen. Hierfür wurde der Linux profiler perf genutzt.

Den Overhead der Graphstruktur wurde gemessen indem eine Beispiel welt ohne Nutzung des Systems nach implementiert wurde.

Neben den reinen Laufzeiteigenschaften spielt auch die Nutzerfreundlichkeit des Systems eine Rolle. Durch den grafischen Editor lassen sich komplexe Abhängigkeiten oft leichter überblicken als in klassischem Quellcode. Gleichzeitig erhöht dieser Ansatz jedoch auch die Komplexität des Systems. Während klassischer Programmcode sehr flexibel ist und ohne zusätzliche Struktur auskommt, erfordert der hier vorgestellte Ansatz eine explizite Modellierung aller Abhängigkeiten im Graphen.

Im dritten sollte die Aufwand zu Erweiterung des Systems betrachtet werden. Meine Implementation nutzt nur simple Operationen auf CSGs. Dies entspricht nicht den wirklichen Datenstrukturen und Problemen in Spielen oder Simulationen. Daher würde eine Große Menge an weiteren Operationen und Datentypen implement werden müssen.

5.1 Reduktion der Neuberechnungszeit

5.2 Overhead

5.3 Nutzerfreundlichkeit

Durch den grafischen Editor kann der Generationsprozess als Abhängigkeits-Graph visualisiert und direkt manipuliert werden. Dadurch lassen sich komplexe Zusammenhänge zwischen einzelnen Operationen häufig leichter nachvollziehen als in klassischem Quellcode, da Datenflüsse und Abhängigkeiten explizit dargestellt sind.

Insbesondere bei der experimentellen Entwicklung von Generationsalgorithmen kann dieser Ansatz Vorteile bieten. Änderungen am Graphen können direkt im Editor vorgenommen werden, ohne dass der gesamte Algorithmus neu implementiert werden muss. In Kombination mit der minimalen Neuberechnung des Generators können Anpassungen am Generationsprozess schnell getestet werden, da nur die betroffenen Teile der Welt neu berechnet werden.

Gleichzeitig hängt die wahrgenommene Nutzerfreundlichkeit stark vom jeweiligen Nutzertyp ab. Nutzer mit viel Erfahrung in klassischer Softwareentwicklung bevorzugen häufig eine direkte Implementierung in Quellcode, da diese mehr Kontrolle bietet und weniger strukturelle Einschränkungen besitzt. Für weniger erfahrene Nutzer oder für Anwender, die primär konzeptionell arbeiten möchten, kann ein grafischer Editor hingegen zugänglicher sein. Die visuelle Darstellung der Abhängigkeiten erleichtert das Verständnis des Systems und erlaubt es, Generationslogik zu definieren, ohne sich stark mit den Details einer Programmiersprache beschäftigen zu müssen. Allerdings kann ein solcher Editor auch einschränkend wirken, da nur die im System vorgesehenen Operationen und Strukturen genutzt werden können.

Insgesamt bietet der grafische Ansatz insbesondere für komplexe prozedurale Systeme Vorteile, da er eine bessere Übersicht über den Generationsprozess ermöglicht und iterative Änderungen am Algorithmus erleichtert, während klassische Code-basierte Ansätze weiterhin mehr Flexibilität für erfahrene Entwickler bieten.

5.4 Erweiterbarkeit

Um das System um weitere Operationen oder oder Datentype zu erweitern ist ein gutes Verständnis der Codestruktur benötigt. Jedoch sind keine grundlegenden Änderungen nötigt. Der Editor, das Template und der Generator stellen keine Erwartungen an die Art der Datentypen oder Operationen im Abhängigkeits Graph. Die Datentypen und Operationen sind mit Typed Unions implement daher können diese einfach erweitert werden. Jedoch muss die neue Variation an jeder Stelle wo ein Datentyp oder Operation allgemein verwendet wird implementiert werden um sie in das bestehende Netz an möglichen Abhängigkeiten zu intrigieren.

6 Future Work

6.1 Nullwerte führen zu leeren Lösungen

Da der Null Wert per definition eine valider Wert ist kann es dazu kommen, dass sich ein Abhängigkeits-Kreis zu null als Lösung entwickelt, auch wenn es theoretisch andere Lösungen gebe. Um dies zu lösen müsste ein anderer Ansatz zur Lösung von Abhängigkeits-Kreisen genutzt werden.

Arbeiten in Richtung closely connected Components in Verbindung könnten hier eine Lösung sein.

6.2 Andere Datenstrukturen

Wie schon in Abschnitt 4.7.4 beschrieben eignen sich CSGs nicht zum direkten rendering. Die Umwandlung zu einem Mesh mit „marching cubes“ ist aufwendig und mehr wesentlich mehr Leistung benötigen, als die Generation selbst. Somit besteht weiterhin die Forschungsfrage in wie weit minimale Neuberechnung von prozeduralen Meshen möglich ist.

6.3 Nutzerfreundlichkeit

- Nutzerfreundlichkeit durch Experten Interviews oder Studien bewerten.

Bibliografie

[1] P. Merrell, „Example-based model synthesis“, in *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*, in I3D '07. Seattle, Washington: Association for Computing Machinery, 2007, S. 105–112. doi: 10.1145/1230100.1230119.

[2] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, und S. Risi, „Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network“. [Online]. Verfügbar unter: https://arxiv.org/abs/1805.00728

[3] M. Bazzaz und S. Cooper, „Level Generation with Constrained Expressive Range“, in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, in FDG '25.: Association for Computing Machinery, 2025. doi: 10.1145/3723498.3723845.

[4] O. Withington und L. Tokarchuk, „Compressing and Comparing the Generative Spaces of Procedural Content Generators“, in *2022 IEEE Conference on Games (CoG)*, 2022, S. 143–150. doi: 10.1109/CoG51982.2022.9893615.

[5] A. Goslin, „Terrain Diffusion: A Diffusion-Based Successor to Perlin Noise in Infinite, Real-Time Terrain Generation“. [Online]. Verfügbar unter: https://arxiv.org/abs/2512.08309

[6] B. Bollobás, *Modern Graph Theory*. Springer New York, 2013. doi: 10.1007/978-1-4612-0619-4.

[7] A. A. G. Requicha und H. B. Voelcker, „Constructive Solid Geometry“, Rochester, NY, USA, technical report, 1977.

[8] B. Wassermann, S. Kollmannsberger, T. Bog, und E. Rank, „From geometric design to numerical analysis: A direct approach using the Finite Cell Method on Constructive Solid Geometry“, *Computers & Mathematics with Applications*, Bd. 74, Nr. 7, S. 1703–1726, Okt. 2017, doi: 10.1016/j.camwa.2017.01.027.

Idee:

Text

ToDo:

Text

TODO: Text

Frage:

Text

$$A = \pi \tag{1}$$